

48_Whack-a-mole

30/09/2026 12:17:44

<https://chatgpt.com/c/6abc8092-8b68-83e8-b9ee-a00f923cd992>



Okay here is the handout for the entire project
Lab 4: VibeCoding
Objective
Use Vibe Coding tools to fix the broken code and add features to the same. Learn how
to prompt to get the job done in under three to four attempts. Clone a repo assigned
to you, make changes and push to your repo in your personal account.
THIS IS AN INDIVIDUAL ASSIGNMENT. WORK INDIVIDUALLY!!
Duration
60 minutes
Software Requirements
● Access to Vibe Coding tool. (ChatGPT, Gemini, Claude etc)
Instructions:
1\. Go to the repo assigned to you
2\. Go through the ReadMe file to understand the deliverables
3\. Clone or Fork the repo
4\. Run the Python based code (A game/ A Puzzle)
5\. Record 10 seconds of video before making any changes.
6\. Write prompts and fix the broken code
7\. Write prompts to add all the features listed in the ReadMe file of the repo one
by one.
8\. Record another 10 seconds of video again after making all the changes.
9\. Push the code to your repo.
10.Do not raise PR to the main repo (SETAPESU26)

Deliverables:

Upload the following to your Lab repo under Lab-4 folder,

- a. Videos (before and after)
- b. Updated Code
- c. Chat history exported as a doc/pdf

Here is the repo am supposed to clone n work on:
https://github.com/SETAPESU26/48_whack-a-mole

Lets start off with it, as usual dont give me any completed files, give me the changes that i'll manually make and incase of any deliverable files give me the content alone which I'll decide

So first the repo inspection

```
PS D:\Akshya\1.Sem_5\SE_Lab\48_whack-a-mole> git status
```

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

```
PS D:\Akshya\1.Sem_5\SE_Lab\48_whack-a-mole> git branch
```

* main

```
PS D:\Akshya\1.Sem_5\SE_Lab\48_whack-a-mole> Get-Childitem
```

Directory: D:\Akshya\1.Sem_5\SE_Lab\48_whack-a-mole

Mode	LastWriteTime	Length	Name
d----	9/30/2026 8:16 AM		game
-a----	9/30/2026 8:16 AM	873	main.py
-a----	9/30/2026 8:16 AM	2835	README.md
-a----	9/30/2026 8:16 AM	8	requirements.txt

```
PS D:\Akshya\1.Sem_5\SE_Lab\48_whack-a-mole> Get-Content Readme.md
```

```
# Whack-a-Mole Game
```

This project is a terminal-based Whack-a-Mole game using **Pygame**. It introduces students to interactive game design using object-oriented principles and real-time graphical rendering.

```
---
```

```
## What's Provided
```

A partially working version of a Whack-a-Mole game with:

- A 3x3 grid of holes that randomly pop moles up for a short time
- Mouse-click "whacking" that scores a point when it lands on an active mole
- A countdown timer and score display

You are expected to **analyze**, **interact with an AI assistant**, and **complete/fix** the game to make it fully functional.

```
### Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.
```

```
---
```

```
## Getting Started
```

```
### Setup
```

1. Clone the repo or download the project folder.
2. Make sure you have Python 3.10+ installed.
3. Install dependencies:

```
```bash
```

```
pip install -r requirements.txt
```
```

4. Run the game:

```
```bash
python main.py
```
```

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Refine Collision Detection

> A single click can sometimes award two points instead of one, when two neighboring moles are both up and the click lands near the border between their holes. Investigate and enhance click-hit accuracy.

Task 2: Implement Game Over Condition

> Add a screen that displays the final score once the round timer reaches zero, then gracefully waits for input instead of just printing to the console.

Task 3: Add Replay Option

> After Game Over, allow the user to play again by choosing a difficulty (Easy, Medium, or Hard mole speed/spawn rate), or exit.

Task 4: Add Sound Feedback

> Add basic sound effects for a successful whack, a missed click, and the round ending.

Expected Behavior

- Moles pop up at random holes for a short time and go back down if not whacked
- Clicking directly on an active mole scores a point and sends it back down immediately
- Clicking an empty hole (or missing entirely) does not score
- A countdown timer and the current score are visible at all times
- The round ends when the timer reaches zero

Folder Structure

```
```\nwhack-a-mole-main/\n  "œ"€"€ main.py\n  "œ"€"€ requirements.txt\n  "œ"€"€ game/\n    ,  "œ"€"€ game_engine.py\n    ,  """€"€ hole.py\n    ""€"€ README.md\n```\n
```

---

### ## Submission Checklist

Submission is only the following three things:

- [] A 10-second video of gameplay **\*\*before\*\*** your changes, showing the bug/broken behavior
  - [] A 10-second video of gameplay **\*\*after\*\*** your changes, showing the bug fixed and the new features working
  - [] The Chat/LLM used page link, with the complete chat history
- PS D:\Akshya\1.Sem\_5\SE\_Lab\48\_whack-a-mole>

Now we r gonna split this into 4 tasks:

1. **\*\*Collision detection\*\*** — prevent one click from scoring twice / improve hit accuracy.
2. **\*\*Game Over\*\*** — proper graphical final-score screen when timer reaches zero.
3. **\*\*Replay + difficulty\*\*** — Easy / Medium / Hard after Game Over.
4. **\*\*Sound feedback\*\*** — successful hit, missed click, round ending.

Lets start with Taks 1, fix collision detection: a single mouse click must never award more than one point, even when neighboring mole hitboxes overlap.

Modify the existing implementation with the smallest reasonable change. A click should register at most one active mole, and clicking an empty area should count as a miss. Preserve the existing game behavior and structure; do not implement the other README tasks yet.

Explain which code you changed and why.

9/30/2026, 8:52:58 AM



Yes — for Task 1, we should keep this **very small and isolated** so your chat history clearly shows an LLM-assisted fix rather than a rewrite.

First, I need to inspect the actual collision code in `game/game_engine.py` (and the hole representation if necessary). Please run:

```
Get-Content game\game_engine.py
```

and

```
Get-Content game\hole.py
```

Paste both outputs here.

Then I'll give you:

1. **Exactly which lines/logic to change**
2. **The replacement code only**, not a completed file
3. A short explanation of **why the current implementation can score twice**
4. A quick test procedure to demonstrate that **one click = at most one point**, including the neighboring-hole case

We'll leave Tasks 2-4 completely untouched.

*9/30/2026, 8:53:01 AM*



```
PS D:\Akshya\1.Sem_5\SE_Lab\48_whack-a-mole> pip
install -r requirements.txt
Collecting pygame (from -r requirements.txt (line 1))
 Downloading pygame-2.6.1-cp312-cp312-
win_amd64.whl.metadata (13 kB)
Downloading pygame-2.6.1-cp312-cp312-
win_amd64.whl (10.6 MB)

10.6/10.6 MB 15.1 MB/s eta 0:00:00
Installing collected packages: pygame
Successfully installed pygame-2.6.1

[notice] A new release of pip is available: 25.0.1 ->
26.2.1
[notice] To update, run: python.exe -m pip install --
upgrade pip
PS D:\Akshya\1.Sem_5\SE_Lab\48_whack-a-mole>
python main.py
pygame 2.6.1 (SDL 2.28.4, Python 3.12.10)
Hello from the pygame community.
https://www.pygame.org/contribute.html
Time's up! Final score: 18
PS D:\Akshya\1.Sem_5\SE_Lab\48_whack-a-mole> Get-
Content main.py
import pygame
from game.game_engine import GameEngine

Initialize pygame/Start application
pygame.init()

Screen dimensions
WIDTH, HEIGHT = 500, 560
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Whack-a-Mole - Pygame
Version")

Colors
GRASS_GREEN = (120, 170, 90)
```



```

Clock
clock = pygame.time.Clock()
FPS = 60

Game loop
engine = GameEngine(WIDTH, HEIGHT)

def main():
 running = True
 while running:
 SCREEN.fill(GRASS_GREEN)
 for event in pygame.event.get():
 if event.type == pygame.QUIT:
 running = False
 engine.handle_event(event)

 engine.handle_input()
 engine.update()
 engine.render(SCREEN)

 pygame.display.flip()
 clock.tick(FPS)

 pygame.quit()

if __name__ == "__main__":
 main()
PS D:\Akshya\1.Sem_5\SE_Lab\48_whack-a-mole> Get-Content game\game_engine.py
import pygame
import random
from .hole import Hole

Game Engine

DARK_BROWN = (60, 40, 20)
MOLE_BROWN = (140, 95, 55)
BLACK = (0, 0, 0)

class GameEngine:
 def __init__(self, width, height, rows=3, cols=3):

```

```

self.width = width
self.height = height

self.holes = []
spacing_x = width // (cols + 1)
spacing_y = (height - 80) // (rows + 1)
for r in range(rows):
 for c in range(cols):
 cx = spacing_x * (c + 1)
 cy = 80 + spacing_y * (r + 1)
 self.holes.append(Hole(cx, cy))

self.spawn_chance = 0.02 # per-hole, per-frame
chance to pop up
self.mole_up_frames = 45 # how long a mole
stays up if not whacked

self.round_seconds = 30
self.time_left_frames = self.round_seconds * 60

self.score = 0
self.misses = 0
self.font = pygame.font.SysFont("Arial", 28)
self.game_over = False

def handle_event(self, event):
 if self.game_over:
 return
 if event.type == pygame.MOUSEBUTTONDOWN:
 self._handle_click(event.pos)

def _handle_click(self, pos):
 hit_something = False

 # NOTE: this loop does not stop after the first hole
 it finds
 # under the click - it checks every hole. Each
 hole's hit-box
 # (Hole.hit_size) is deliberately a bit larger than
 the
 # spacing between holes, so neighboring hit-

```

```

boxes overlap
 # slightly near the grid lines. If two adjacent
 moles happen
 # to both be up and the player clicks in that
 overlap zone,
 # both holes register a hit from the same click,
 awarding two
 # points for a single whack. See Task 1 in the
 README.
 for hole in self.holes:
 if hole.rect().collidepoint(pos):
 if hole.whack():
 self.score += 1
 hit_something = True

 if not hit_something:
 self.misses += 1

 def handle_input(self):
 # Reserved for continuously-held-key input; this
 game is
 # entirely mouse-driven, so there's nothing to poll
 here.
 pass

 def update(self):
 if self.game_over:
 return

 self.time_left_frames -= 1
 if self.time_left_frames <= 0:
 self.game_over = True
 return

 for hole in self.holes:
 hole.update()
 if not hole.active and random.random() <
self.spawn_chance:
 hole.pop_up(self.mole_up_frames)

 def render(self, screen):

```

```

 for hole in self.holes:
 pygame.draw.circle(screen, DARK_BROWN,
 (hole.center_x, hole.center_y), 40)
 if hole.active:
 pygame.draw.circle(screen, MOLE_BROWN,
 (hole.center_x, hole.center_y), 32)

 score_text = self.font.render(f"Score:
{self.score}", True, BLACK)
 screen.blit(score_text, (10, 10))

 seconds_left = max(0, self.time_left_frames // 60)
 timer_text = self.font.render(f"Time:
{seconds_left}s", True, BLACK)
 screen.blit(timer_text, (self.width - 140, 10))

 if self.game_over and not getattr(self,
 "_game_over_logged", False):
 # NOTE: no proper game-over screen yet - see
 Task 2 in the README.
 print("Time's up! Final score:", self.score)
 self._game_over_logged = True
PS D:\Akshya\1.Sem_5\SE_Lab\48_whack-a-mole> Get-
Content game\hole.py
import pygame

class Hole:
 def __init__(self, center_x, center_y, hit_size=150):
 self.center_x = center_x
 self.center_y = center_y
 # NOTE: intentionally larger than the spacing
 between holes -
 # see Task 1 in the README.
 self.hit_size = hit_size
 self.active = False
 self.timer = 0

 def pop_up(self, duration_frames):
 self.active = True
 self.timer = duration_frames

```

```
def update(self):
 if self.active:
 self.timer -= 1
 if self.timer <= 0:
 self.active = False

def whack(self):
 was_active = self.active
 if was_active:
 self.active = False
 self.timer = 0
 return was_active

def rect(self):
 return pygame.Rect(
 self.center_x - self.hit_size // 2,
 self.center_y - self.hit_size // 2,
 self.hit_size,
 self.hit_size,
)
```

PS D:\Akshya\1.Sem\_5\SE\_Lab\48\_whack-a-mole>

Okay now for all changes in code let me know where and I'll manually copy paste it

9/30/2026, 8:55:23 AM



Perfect. We can fix **Task 1 with a one-function change**. No need to touch `main.py`, `hole.py`, or any other task.

## Task 1 — Collision Detection

The problem is exactly here in `game/game_engine.py`:

```
def _handle_click(self, pos):
```

Currently, the loop checks **every hole**, so if the click falls inside two overlapping rectangles and both moles are active, both `whack()` calls succeed.

## Change only `_handle_click()`

In `game/game_engine.py`, replace the **entire existing** `_handle_click()` **method**:

```
def _handle_click(self, pos):
 hit_something = False

 # NOTE: this loop does not stop after the first hole it finds
 # under the click - it checks every hole. Each hole's hit-box
 # (Hole.hit_size) is deliberately a bit larger than the
 # spacing between holes, so neighboring hit-boxes overlap
 # slightly near the grid lines. If two adjacent moles happen
 # to both be up and the player clicks in that overlap zone,
 # both holes register a hit from the same click, awarding two
 # points for a single whack. See Task 1 in the README.
 for hole in self.holes:
 if hole.rect().collidepoint(pos):
 if hole.whack():
 self.score += 1
 hit_something = True

 if not hit_something:
 self.misses += 1
```

with this:

```
def _handle_click(self, pos):
 for hole in self.holes:
 if hole.rect().collidepoint(pos) and hole.active:
 hole.whack()
 self.score += 1
 return

 self.misses += 1
```

## Why this fixes it

The important change is:

```
return
```

After the first active mole is successfully hit:

```
click
 ↓
check holes
 ↓
active mole found
 ↓
whack it
 ↓
score += 1
 ↓
RETURN immediately
```

So even if the click position is inside another neighboring mole's hitbox, **the same mouse click cannot continue through the loop and score again.**

It also handles misses cleanly:

```
self.misses += 1
```

is reached only if **no active mole** was hit.

## Why I'm not changing `Hole.hit_size`

You might notice that `hit_size=150` is deliberately larger than the spacing between holes. We **shouldn't change that** for this task.

The README specifically asks us to handle the case where hitboxes overlap. Making the hitboxes smaller would change the game's existing interaction design. Stopping after the first successful hit is the smallest reasonable fix while preserving the existing structure.

---

## Test it

Save the file and run:

```
python main.py
```

For your Task 1 testing, verify:

1. Click directly on an active mole → **score increases by 1.**
2. Click an empty area → **score doesn't increase.**
3. If you can get two neighboring moles active simultaneously, click their overlapping region → **score increases by only 1.**
4. The hit mole disappears immediately.
5. The rest of the game behaves exactly as before.

You can also check the change before moving on:

```
git diff -- game/game_engine.py
```

That diff is useful for your **LLM chat-history evidence**, because it clearly shows that Task 1 involved a small targeted modification rather than rewriting the game.

**Don't commit yet** if you want to keep the four tasks separately traceable. After we've finished Task 1 and verified it, we can move to Task 2.

*9/30/2026, 8:55:25 AM*





Task 1 is done and verified go to Task 2

When the round timer reaches zero, the game should display a proper graphical Game Over screen in the Pygame window showing:

- "Game Over"
- the final score
- a clear instruction telling the player to press a key or mouse button to continue/exit

The game should stop accepting normal mole-click gameplay once the round ends, and it should no longer rely on printing the final score to the console.

Keep the existing game structure and Task 1 collision fix intact. Do not implement the replay/difficulty or sound features yet.

9/30/2026, 8:58:18 AM



Great. Task 1 stays exactly as it is. For **Task 2**, we only need to change the game-over state handling and rendering. We will **not** add replay, difficulty, or sound yet.

There are two files to modify:

- `game/game_engine.py`
- `main.py`

The key idea is: once `time_left_frames` reaches zero, `game_over` becomes `True`; normal mole clicks are already blocked by Task 1's existing `handle_event()` guard. We then render a proper Game Over screen and let a key/mouse click exit the game.

## 1. Modify `game/game_engine.py`

### A. Add an exit flag in `__init__`

Find:

```
self.game_over = False
```

Immediately **below it**, add:

```
self.exit_requested = False
```

So that section becomes:

```
self.font = pygame.font.SysFont("Arial", 28)
self.game_over = False
self.exit_requested = False
```

---

## B. Replace `handle_event()`

Find the current:

```
def handle_event(self, event):
 if self.game_over:
 return
 if event.type == pygame.MOUSEBUTTONDOWN:
 self._handle_click(event.pos)
```

Replace it with:

```
def handle_event(self, event):
 if self.game_over:
 if event.type == pygame.KEYDOWN or event.type ==
pygame.MOUSEBUTTONDOWN:
 self.exit_requested = True
 return

 if event.type == pygame.MOUSEBUTTONDOWN:
 self._handle_click(event.pos)
```

## Why?

Before Game Over:

mouse click → normal mole gameplay

After Game Over:

key/mouse click → exit game

So a mouse click on the Game Over screen **cannot accidentally whack a mole**.

### C. Replace the Game Over section in `render()`

At the bottom of `render()`, you currently have:

```
if self.game_over and not getattr(self, "_game_over_logged", False):
 # NOTE: no proper game-over screen yet - see Task 2 in the README.
 print("Time's up! Final score:", self.score)
 self._game_over_logged = True
```

Delete that entire block and replace it with:

```
if self.game_over:
 overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
 overlay.fill((0, 0, 0, 150))
 screen.blit(overlay, (0, 0))

 game_over_text = self.font.render("Game Over", True, (255, 255, 255))
 score_text = self.font.render(
 f"Final Score: {self.score}", True, (255, 255, 255)
)
 instruction_text = self.font.render(
 "Press any key or mouse button to exit", True, (255, 255, 255)
)

 screen.blit(
 game_over_text,
 game_over_text.get_rect(center=(self.width // 2, self.height // 2 -
60)),
)
 screen.blit(
 score_text,
 score_text.get_rect(center=(self.width // 2, self.height // 2)),
```

```

)
 screen.blit(
 instruction_text,
 instruction_text.get_rect(center=(self.width // 2, self.height // 2 +
60)),
)

```

This gives you a proper graphical overlay rather than printing:

```

Game Over

Final Score: 18

Press any key or mouse button to exit

```

The underlying game screen remains visible but is darkened by the transparent overlay.

## 2. Modify `main.py`

We need one tiny change so that the main game loop actually exits when the player responds to the Game Over screen.

Find:

```

for event in pygame.event.get():
 if event.type == pygame.QUIT:
 running = False
 engine.handle_event(event)

```

Replace it with:

```

for event in pygame.event.get():
 if event.type == pygame.QUIT:
 running = False

 engine.handle_event(event)

 if engine.exit_requested:
 running = False

```

That's all we need in `main.py`.

---

## What happens now

The complete flow becomes:

```
Game starts
 ↓
Moles appear and can be clicked
 ↓
Timer reaches 0
 ↓
game_over = True
 ↓
Normal mole gameplay stops
 ↓
Game Over screen appears
 ↓
Final score displayed
 ↓
"Press any key or mouse button to exit"
 ↓
Player presses key/clicks
 ↓
exit_requested = True
 ↓
main loop ends
 ↓
pygame.quit()
```

And importantly, **we are not implementing replay yet**. Task 3 will later change what happens after this screen.

## Verify Task 2

Run:

```
python main.py
```

You should verify:

- The timer reaches `0`.
- A graphical **Game Over** screen appears.
- The final score is displayed.
- There is an instruction to press a key/mouse button.
- Clicking where a mole used to be does **not** increase the score.
- Pressing a key exits.
- Clicking the mouse exits.
- The console no longer prints `Time's up! Final score: ...`.

Then inspect both changes:

```
git diff -- game/game_engine.py main.py
```

One important thing for your lab evidence: **don't modify anything for Task 3 yet**. Once you've verified this, we can move to **Task 3: Replay + Easy/Medium/Hard difficulty**, building directly on this Game Over state.

*9/30/2026, 8:58:20 AM*



Verified now for task 3

After Game Over, display a menu that allows the player to:

- start a new game with Easy difficulty
- start a new game with Medium difficulty
- start a new game with Hard difficulty
- exit the game

The three difficulties should change the mole speed/spawn rate so that Easy is slower, Medium is the current/default difficulty, and Hard is faster/more frequent.

When replaying, reset the score, timer, mole states, and other round-specific state so the new round starts cleanly.

Keep the existing Task 1 collision fix, Task 2 Game Over screen, and overall project structure intact. Do not implement sound yet.

9/30/2026, 8:58:50 AM



Perfect. For Task 3, we can build directly on the Task 2 state machine without restructuring the project.

The cleanest approach is:

- **Game Over state** → show difficulty menu.
- `1` → Easy
- `2` → Medium
- `3` → Hard
- `Esc` → Exit
- Selecting a difficulty calls a new `reset_round()` method.

- The new round resets **score, timer, mole states, and timers**.
- Task 1 collision logic remains untouched.
- Task 2's Game Over behavior remains intact.
- No sound yet.

We only need to modify `game/game_engine.py` and make a small change to `main.py`.

## 1. Modify `game/game_engine.py`

### A. Add difficulty settings in `__init__`

Find:

```
self.spawn_chance = 0.02
self.mole_up_frames = 45
```

Replace those two lines with:

```
self.difficulties = {
 "Easy": (0.01, 60),
 "Medium": (0.02, 45),
 "Hard": (0.04, 30),
}

self.difficulty = "Medium"
self.spawn_chance, self.mole_up_frames = self.difficulties[self.difficulty]
```

So the behavior is:

| Difficulty | Spawn chance | Mole stays up |
|------------|--------------|---------------|
| Easy       | 0.01         | 60 frames     |
| Medium     | 0.02         | 45 frames     |
| Hard       | 0.04         | 30 frames     |

Since the game runs at 60 FPS:



- **Easy:** slower spawning and moles remain longer
- **Medium:** current/default behavior
- **Hard:** more frequent spawning and moles disappear faster

This gives you a clear, demonstrable difficulty difference without changing the game's basic mechanics.

---

## 2. Add `reset_round()`

In `game/game_engine.py`, add this method **after** `__init__()` **and before** `handle_event()`:

```
def reset_round(self, difficulty):
 self.difficulty = difficulty
 self.spawn_chance, self.mole_up_frames = self.difficulties[difficulty]

 self.time_left_frames = self.round_seconds * 60
 self.score = 0
 self.misses = 0
 self.game_over = False
 self.exit_requested = False

 for hole in self.holes:
 hole.active = False
 hole.timer = 0
```

### Why this is important

We don't want a replay to simply set:

```
self.game_over = False
```

because the previous round could still have:

- an old score
- expired timer
- active moles

- mole countdown timers
- previous difficulty

`reset_round()` gives us a genuinely clean round.

---

### 3. Modify `handle_event()`

You currently have the Task 2 version:

```
def handle_event(self, event):
 if self.game_over:
 if event.type == pygame.KEYDOWN or event.type ==
pygame.MOUSEBUTTONDOWN:
 self.exit_requested = True
 return

 if event.type == pygame.MOUSEBUTTONDOWN:
 self._handle_click(event.pos)
```

Replace it with:

```
def handle_event(self, event):
 if self.game_over:
 if event.type == pygame.KEYDOWN:
 if event.key == pygame.K_1:
 self.reset_round("Easy")
 elif event.key == pygame.K_2:
 self.reset_round("Medium")
 elif event.key == pygame.K_3:
 self.reset_round("Hard")
 elif event.key == pygame.K_ESCAPE:
 self.exit_requested = True

 elif event.type == pygame.MOUSEBUTTONDOWN:
 self.exit_requested = True

 return

 if event.type == pygame.MOUSEBUTTONDOWN:
 self._handle_click(event.pos)
```

## Important behavior

When Game Over is active:

```
1 → Easy → new round
2 → Medium → new round
3 → Hard → new round
Esc → exit
```

A mouse click on the Game Over/menu screen still exits, preserving the Task 2 behavior.

Most importantly, because we `return` while `self.game_over` is true, the click **cannot reach** `_handle_click()`.

So your Task 1 protection remains intact.

---

## 4. Change the Game Over display into a Replay menu

Now go to the bottom of `render()`.

You currently have the Task 2 block beginning with:

```
if self.game_over:
```

Replace that **entire Game Over rendering block** with:

```
if self.game_over:
 overlay = pygame.Surface((self.width, self.height),
pygame.SRCALPHA)
 overlay.fill((0, 0, 0, 150))
 screen.blit(overlay, (0, 0))

 game_over_text = self.font.render("Game Over", True, (255, 255,
255))

 score_text = self.font.render(
 f"Final Score: {self.score}", True, (255, 255, 255)
)
 menu_text = self.font.render(
 "Choose a difficulty:", True, (255, 255, 255)
)
```

```

easy_text = self.font.render(
 "1 - Easy", True, (255, 255, 255)
)
medium_text = self.font.render(
 "2 - Medium", True, (255, 255, 255)
)
hard_text = self.font.render(
 "3 - Hard", True, (255, 255, 255)
)
exit_text = self.font.render(
 "Esc - Exit", True, (255, 255, 255)
)

screen.blit(
 game_over_text,
 game_over_text.get_rect(
 center=(self.width // 2, self.height // 2 - 130)
),
)
screen.blit(
 score_text,
 score_text.get_rect(
 center=(self.width // 2, self.height // 2 - 80)
),
)
screen.blit(
 menu_text,
 menu_text.get_rect(
 center=(self.width // 2, self.height // 2 - 30)
),
)
screen.blit(
 easy_text,
 easy_text.get_rect(
 center=(self.width // 2, self.height // 2 + 20)
),
)
screen.blit(
 medium_text,
 medium_text.get_rect(
 center=(self.width // 2, self.height // 2 + 55)
),
)

```

```

),
)
 screen.blit(
 hard_text,
 hard_text.get_rect(
 center=(self.width // 2, self.height // 2 + 90)
),
)
 screen.blit(
 exit_text,
 exit_text.get_rect(
 center=(self.width // 2, self.height // 2 + 125)
),
)

```

The screen will now look roughly like:

```

Game Over

Final Score: 18

Choose a difficulty:

1 - Easy
2 - Medium
3 - Hard
Esc - Exit

```

## 5. `main.py`

Good news: **you don't need another structural change to** `main.py`.

Keep the Task 2 version:

```

for event in pygame.event.get():
 if event.type == pygame.QUIT:
 running = False

engine.handle_event(event)

```

```
if engine.exit_requested:
 running = False
```

When you choose 1, 2, or 3, `reset_round()` sets:

```
self.game_over = False
self.exit_requested = False
```

so the existing main loop automatically continues into the new round.

---

## 6. One small thing to verify

Your `update()` already contains:

```
if self.game_over:
 return
```

**Do not change this.**

That's exactly what we want.

During Game Over:

```
game_over = True
↓
update() returns immediately
↓
moles don't move/spawn
↓
menu remains displayed
```

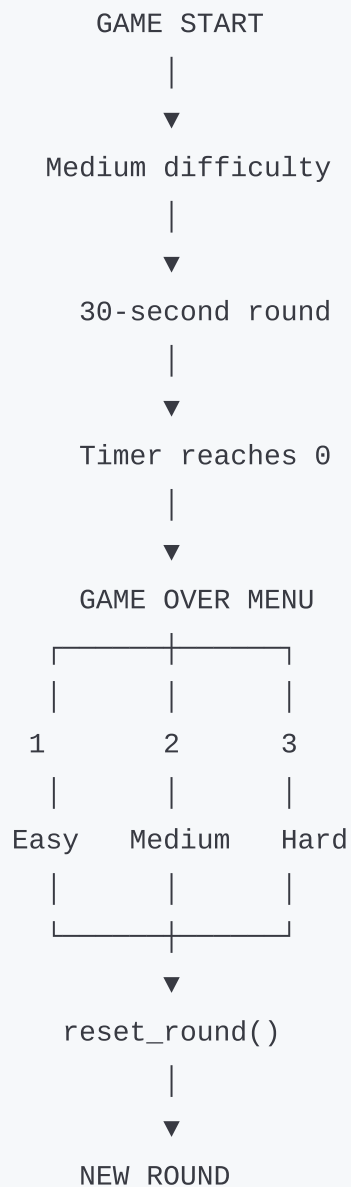
After selecting a difficulty:

```
reset_round()
↓
game_over = False
↓
update() runs normally again
```

---

## Task 3 flow

You should now have:



And:

Esc → exit

### Verify Task 3

Run:

```
python main.py
```

Let the round finish and test each option separately.

For example:

## Test Easy

```
Game Over
↓
Press 1
↓
Score resets to 0
Timer resets to 30s
No old moles remain active
Moles spawn more slowly
```

## Test Medium

```
Game Over
↓
Press 2
↓
Score = 0
Timer = 30s
Default spawn behavior returns
```

## Test Hard

```
Game Over
↓
Press 3
↓
Score = 0
Timer = 30s
Moles appear more frequently and stay up for less time
```

Then test:

```
Game Over → Esc → program exits
```

And importantly, test that **Task 1 still works in the new rounds**: one click on an overlapping pair must still produce at most one point.

Once you've verified it, run:



```
git diff -- game/game_engine.py main.py
```

That will give you a nice Task 3 change record for the eventual chat-history submission. After that, we're down to the final feature: **Task 4 — sound feedback**, without disturbing the first three tasks.

*9/30/2026, 8:58:53 AM*



Verified now go to task 4 Add basic sound feedback for:

1. a successful mole whack
2. a missed click
3. the round ending when the timer reaches zero

Use Pygame's existing audio functionality and avoid requiring external sound files if possible. The sounds should be short and clearly distinguishable.

Initialize and clean up the audio system appropriately, and make sure the game still works if audio initialization is unavailable.

Keep Tasks 1, 2, and 3 working exactly as they are. Do not change the collision detection, Game Over flow, replay menu, or difficulty behavior except where necessary to trigger the sounds.

9/30/2026, 8:59:24 AM



Yep. For Task 4, we can keep the first three tasks intact and add audio as a **best-effort layer**. Since you specifically want no external sound files, we'll synthesize three short tones using `pygame.mixer.Sound(buffer=...)`.

We'll touch only `main.py` and `game/game_engine.py`.

## 1. Modify `main.py` — initialize and clean up audio

At the top, you currently have:

```
pygame.init()
```

Change it to:

```
pygame.init()

audio_available = False
```

```
try:
 pygame.mixer.init()
 audio_available = True
except pygame.error:
 pass
```

This means the game continues normally even if the machine cannot initialize an audio device.

## Then change GameEngine creation

Find:

```
engine = GameEngine(WIDTH, HEIGHT)
```

Change it to:

```
engine = GameEngine(WIDTH, HEIGHT, audio_available)
```

## Finally, clean up the mixer

At the bottom of `main()`, you currently have:

```
pygame.quit()
```

Change it to:

```
if audio_available:
 pygame.mixer.quit()

pygame.quit()
```

So the audio system is explicitly cleaned up.

---

## 2. Modify `game/game_engine.py`

### A. Change the constructor

Find:

```
def __init__(self, width, height, rows=3, cols=3):
```

Change it to:

```
def __init__(self, width, height, audio_available=False, rows=3, cols=3):
```

This keeps the existing parameters working while allowing `main.py` to tell the engine whether audio is available.

---

## B. Add the sound setup

Inside `__init__`, after:

```
self.exit_requested = False
```

add:

```
self.audio_available = audio_available
self.hit_sound = None
self.miss_sound = None
self.game_over_sound = None

if self.audio_available:
 self.hit_sound = self._create_tone(700, 100)
 self.miss_sound = self._create_tone(220, 120)
 self.game_over_sound = self._create_tone(120, 400)
```

The three sounds are intentionally different:

- **Hit:** short, higher-pitched tone
  - **Miss:** short, lower-pitched tone
  - **Game Over:** longer, low tone
- 

## C. Add a helper for generating the tones

Still inside `GameEngine`, add this method **after** `__init__()` **and before** `reset_round()`:

```
def _create_tone(self, frequency, duration_ms):
 sample_rate = 44100
 sample_count = int(sample_rate * duration_ms / 1000)

 buffer = bytearray()
 amplitude = 8000

 for i in range(sample_count):
 value = int(
 amplitude
 * pygame.math.sin(2 * pygame.math.pi * frequency * i /
sample_rate)
)

 buffer.extend(value.to_bytes(2, byteorder="little", signed=True))

 return pygame.mixer.Sound(buffer=bytes(buffer))
```

**One correction:** Pygame's `pygame.math` does not provide `sin()`. So we need Python's standard `math` module.

At the very top of `game_engine.py`, you currently have:

```
import pygame
import random
```

Change it to:

```
import pygame
import random
import math
```

Then use this version of `_create_tone()`:

```
def _create_tone(self, frequency, duration_ms):
 sample_rate = 44100
 sample_count = int(sample_rate * duration_ms / 1000)

 buffer = bytearray()
 amplitude = 8000
```

```

for i in range(sample_count):
 value = int(
 amplitude
 * math.sin(2 * math.pi * frequency * i / sample_rate)
)

 buffer.extend(value.to_bytes(2, byteorder="little", signed=True))

return pygame.mixer.Sound(buffer=bytes(buffer))

```

### 3. Add the successful-whack sound

This is where we need to preserve your **Task 1 collision logic exactly**.

Your current `_handle_click()` should be:

```

def _handle_click(self, pos):
 for hole in self.holes:
 if hole.rect().collidepoint(pos) and hole.active:
 hole.whack()
 self.score += 1
 return

 self.misses += 1

```

Do **not** change the collision logic.

Just add the sound immediately after the score increment:

```

def _handle_click(self, pos):
 for hole in self.holes:
 if hole.rect().collidepoint(pos) and hole.active:
 hole.whack()
 self.score += 1

 if self.hit_sound:
 self.hit_sound.play()

 return

```

```
self.misses += 1
```

The important part is that the `return` stays exactly where it is.

Therefore:

```
one click
 ↓
one active mole
 ↓
score +1
 ↓
hit sound
 ↓
return
```

The sound does **not** alter Task 1's one-click/one-point behavior.

## 4. Add the missed-click sound

At the bottom of `_handle_click()` you now have:

```
self.misses += 1
```

Change it to:

```
self.misses += 1

if self.miss_sound:
 self.miss_sound.play()
```

So the complete function is:

```
def _handle_click(self, pos):
 for hole in self.holes:
 if hole.rect().collidepoint(pos) and hole.active:
 hole.whack()
 self.score += 1

 if self.hit_sound:
```

```
 self.hit_sound.play()

 return

 self.misses += 1

 if self.miss_sound:
 self.miss_sound.play()
```

That's the only gameplay change here.

---

## 5. Add the Game Over sound

Now go to `update()`.

You currently have:

```
if self.time_left_frames <= 0:
 self.game_over = True
 return
```

Change it to:

```
if self.time_left_frames <= 0:
 self.game_over = True

 if self.game_over_sound:
 self.game_over_sound.play()

 return
```

This is important because the sound is triggered **exactly once when the timer transitions into Game Over**.

Your existing:

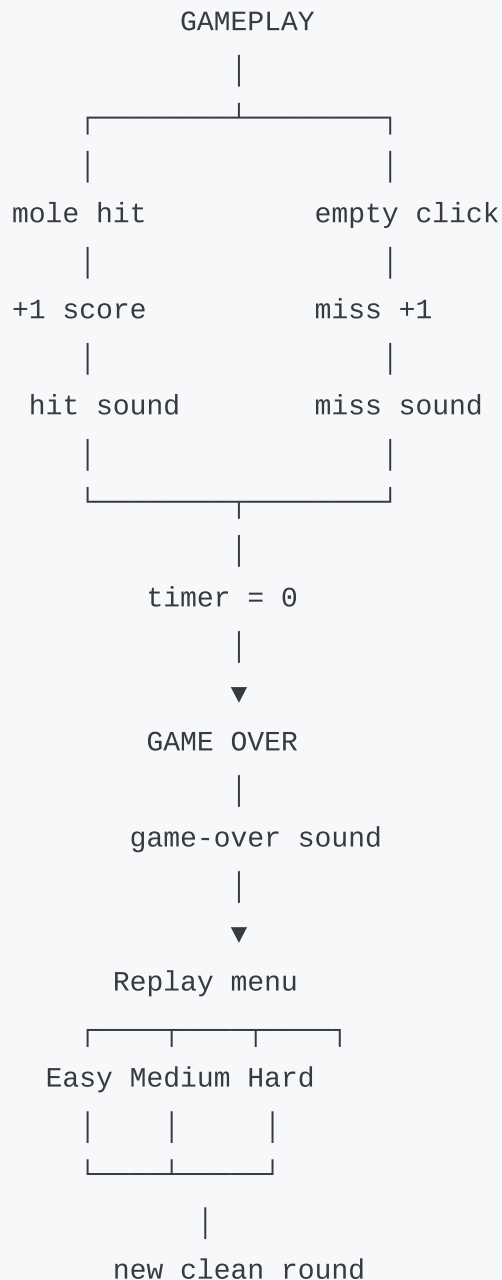
```
if self.game_over:
 return
```

at the beginning of `update()` prevents the Game Over sound from being played repeatedly on every frame.



## 6. What we have now

Your four tasks now work together:



And `Esc` still exits.

### One important implementation detail

The generated sounds don't require `.wav`, `.mp3`, or any other external files. They're created in memory when the game starts.

If audio initialization fails:

```
try:
 pygame.mixer.init()
 audio_available = True
except pygame.error:
 pass
```

then:

```
audio_available = False
```

and the sound objects remain `None`.

The game therefore still functions as a normal Whack-a-Mole game with Tasks 1–3 intact.

---

## Verify Task 4

Run:

```
python main.py
```

Test these four things:

### Successful whack

Click an active mole.

Expected:

```
mole disappears
score +1
short high tone
```

### Miss

Click an empty hole/background.

Expected:

```
score unchanged
miss count increases internally
```

```
short low tone
```

## Game Over

Let the timer reach zero.

Expected:

```
Game Over menu appears
final score displayed
longer low tone
```

## Replay

Select:

```
1 → Easy
2 → Medium
3 → Hard
```

and verify the replay behavior still works exactly as before.

Also test the collision edge case from Task 1 once more: **two neighboring active moles + overlapping hitboxes + one click = only one point and one hit sound.**

Finally:

```
git diff -- game/game_engine.py main.py
```

At this point, your four implementation tasks are complete. The next thing I'd do **before pushing anything** is run one final end-to-end check and inspect `git diff` so we can make sure no accidental changes slipped into the repo.

*9/30/2026, 8:59:25 AM*



Okay now we need to also push the before n after video....n the chat history of new chat as doc or pdf

9/30/2026, 11:48:51 AM



Yep. Now that the implementation is finished, we should handle the **submission packaging** carefully so your Lab repo contains exactly what the handout asks for.

Your final `Lab-4` folder should contain:

```
Lab-4/
├── before/
│ └── before.mp4
├── after/
│ └── after.mp4
└── Chat-History.pdf (or .docx)
```

The **updated code itself should also be in the Lab repo**, according to the broader handout, but we should first make sure your game repo is pushed correctly.

## 1. First verify the game repo

From:

```
D:\Akshya\1.Sem_5\SE_Lab\48_whack-a-mole
```

run:

```
git status
```

Then:

```
git diff
```

Don't push yet if you see anything unexpected.

You should have changes only in:

```
main.py
game/game_engine.py
```

Your Task 1-4 changes should all be there.

---

## 2. Commit the game changes

Once `git diff` looks correct:

```
git add main.py game\game_engine.py
```

Then:

```
git commit -m "Complete Whack-a-Mole lab tasks"
```

Then:

```
git push origin main
```

This pushes your completed game to **your personal repo**, not the `SETAPESU26` repository.

Afterward:

```
git status
```

You want:

```
nothing to commit, working tree clean
```

---

## 3. Before video

You already ran the original game before making changes and demonstrated the broken behavior. If you have that recording, keep it as your **before video**.

I'd name it:

```
before.mp4
```

The important thing is that it demonstrates the **original/broken state**, ideally showing the double-score collision issue.

Don't edit it heavily. A straightforward ~10-second recording is better for this lab because the instructor specifically asked for before/after evidence.

---

## 4. After video

Now record a fresh ~10-second video from your completed game.

I'd recommend demonstrating multiple features quickly:

```
0-3 sec → gameplay + successful mole whack
3-5 sec → another interaction / miss
5-8 sec → Game Over screen
8-10 sec → select a difficulty and show replay starting
```

If getting all of that into exactly 10 seconds is awkward, prioritize:

**Game Over → replay/difficulty → new round**

because Tasks 1 and 4 can be demonstrated during normal gameplay.

Name it:

```
after.mp4
```

---

## 5. Where to put the videos

This part depends on what your **Lab submission repo** is called.

You currently have the actual game repo here:

```
D:\Akshya\1.Sem_5\SE_Lab\48_whack-a-mole
```

But the handout says:

*Upload the following to your Lab repo under Lab-4 folder.*

So **don't put the videos into** `48_whack-a-mole` **unless that is also your Lab submission repo.**

If you have a separate Lab repository, the structure should be:

```
Your-Lab-Repo/
|
├─ Lab-1/
├─ Lab-2/
├─ Lab-3/
├─ Lab-4/
| ├─ before/
| | └─ before.mp4
| ├─ after/
| | └─ after.mp4
| └─ Chat-History.pdf
|
└─ ...
```

If the instructor specifically wants the **updated code under** Lab-4 **too**, then use:

```
Lab-4/
├─ before/
| └─ before.mp4
├─ after/
| └─ after.mp4
├─ Chat-History.pdf
└─ 48_whack-a-mole/
 ├─ main.py
 ├─ requirements.txt
 ├─ README.md
 └─ game/
 ├─ game_engine.py
 └─ hole.py
```

## Important

**Don't duplicate the entire game repo yet.** Tell me the name/path of your Lab submission repo and I'll give you the exact PowerShell commands to place the videos and code correctly.

---

## 6. Chat history

For the chat-history deliverable, you have two reasonable choices:

## Option A — Export this ChatGPT conversation

Since the assignment explicitly asks for the **complete chat history**, this is the safest option.

Export/download this conversation from ChatGPT and save it as:

Chat-History.pdf

That preserves the actual interaction, including:

- Task 1 discussion
- Task 1 code change
- Task 2 discussion
- Task 2 code change
- Task 3 discussion
- Task 3 code change
- Task 4 discussion
- Task 4 code change
- verification steps

This is better evidence than manually reconstructing the conversation.

## Option B — Make a formatted document

If your instructor specifically wants a document rather than the platform's conversation export, I can prepare the **complete content for the chat-history document** based on this conversation.

I'd structure it as:

LAB 4 – VIBECODING

WHACK-A-MOLE

Student: Akshya



Repository: 48\_whack-a-mole

=====

## 1. REPOSITORY INSPECTION

=====

[User prompt]

[LLM response]

=====

## 2. TASK 1 - COLLISION DETECTION

=====

[User prompt]

[LLM response]

Verification:

Task 1 verified successfully.

=====

## 3. TASK 2 - GAME OVER

=====

[User prompt]

[LLM response]

Verification:

Task 2 verified successfully.

=====

## 4. TASK 3 - REPLAY + DIFFICULTY

=====

[User prompt]

[LLM response]

Verification:

Task 3 verified successfully.

=====

5. TASK 4 – SOUND FEEDBACK

=====

[User prompt]

[LLM response]

Verification:

Task 4 verified successfully.

=====

6. FINAL IMPLEMENTATION

=====

Task 1 – Collision Detection

Task 2 – Game Over

Task 3 – Replay + Difficulty

Task 4 – Sound Feedback

=====

7. FINAL VERIFICATION

=====

Game tested successfully.

But **don't manually create that yet** if the assignment really means *complete chat history*. The actual conversation export is preferable.

## One thing I want to check before giving you the exact final commands

Run this from your `SE_Lab` directory:

```
Get-ChildItem
```

and paste the output.

I want to see your **actual Lab submission repository/folder** so I can tell you exactly where `before.mp4` , `after.mp4` , the chat history, and the completed game code should go without accidentally putting them in the wrong Git repository.

*9/30/2026, 11:48:53 AM*